Task 1 — Spring Boot Microservices Foundation

Step 2 — Understand Spring Boot

Spring Framework

Spring is a Java framework used to build applications.

It helps developers create applications without writing a lot of repetitive code.

Spring Boot

Spring Boot is built on top of Spring Framework.

It makes Spring development much easier.

Example, traditionally you might have to configure many things manually.

Spring Boot provides:**Auto-configuration + embedded server + starter dependencies.**

Dependency Injection

Suppose you have:

Controller

↓

Service

↓

Repository

The Controller needs the Service.

Instead of manually creating the Service:

Service service = new Service();

Spring can create and provide it automatically.

That is **Dependency Injection (DI)**.

IoC — Inversion of Control

IoC means **Spring takes control of creating and managing objects**.

Normally:

Developer

↓

Creates object

↓

Uses object

With Spring:

Spring Container

↓

Creates object

↓

Manages object

↓

Provides object to your class

Simple difference

IoC = Spring controls object creation.

Dependency Injection = Spring gives those objects to the classes that need them.

Spring Boot Starter

A Spring Boot Starter is a convenient collection of dependencies.

Example: spring-boot-starter-web

gives you the libraries required to build a web application.

Instead of adding many individual dependencies, you add one starter.

Example in pom.xml:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Auto Configuration

This is one of the biggest advantages of Spring Boot.

Spring Boot looks at the dependencies in your project and automatically configures many things.

Example:

spring-boot-starter-web

↓

Spring Boot understands:

"This is a web application"

↓

Configures required components

↓

Application can run

You don't have to manually configure everything.

Embedded Server

Spring Boot normally comes with an embedded web server such as **Tomcat** when using the web starter.

That means you don't need to separately install and configure a server just to run your basic application.

we can run:

Spring Boot Application

↓

Embedded Tomcat

↓

http://localhost:8080

Step 3 — Create the Spring Boot Project

create a Spring Boot project using **Spring Initializr**.

You can open:

Spring Initializr

Select these values

Field	Value
Project	Maven
Language	Java
Spring Boot	Use the current stable version
Group	com.company
Artifact	userservice
Name	userservice
Packaging	Jar
Java	17 or 21

Dependencies

Select:

1. **Spring Web**
2. **Spring Data JPA**
3. **Validation**
4. **PostgreSQL Driver** — if using PostgreSQL
5. **Lombok**
6. **Spring Boot Actuator**

If your task specifically requires MySQL, select **MySQL Driver** instead of PostgreSQL Driver.

Then click:

Generate → Download ZIP → Extract → Open in Eclipse/IntelliJ/VS Code

Step 4 — Create Package Structure

com.company.userservice

├── controller

├── service

├── repository

├── entity

├── dto

├── exception

├── config

└── UserServiceApplication.java

What does each package mean:

Package	Purpose
controller	Receives HTTP requests
service	Contains business logic
repository	Communicates with database
entity	Database table/model
dto	Data sent/received through API
exception	Handles errors
config	Application configuration
Main class	Starts Spring Boot

In your IDE, create these packages under:

src/main/java/com/company/userservice

Step 5 — Create User Entity

Create User.java

Go to:

src/main/java

└── com.company.userservice

└── entity

└── User.java

code:

```
package com.company.userservice.entity;

import jakarta.persistence.*;

import lombok.AllArgsConstructor;

import lombok.Data;

import lombok.NoArgsConstructor;

import java.time.LocalDateTime;

@Entity

@Table(name = "users")

@Data

@NoArgsConstructor

@AllArgsConstructor

public class User {

    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private Long id;

    @Column(nullable = false)

    private String name;
```

```
    @Column(nullable = false, unique = true)
    private String email;
    @Column(nullable = false)
    private String phone;
   @Column(nullable = false)
    private String password;
    @Column(nullable = false)
    private String status;
    @Column(nullable = false, updatable = false)
    private LocalDateTime createdAt;
   @Column(nullable = false)
    private LocalDateTime updatedAt;
    @PrePersist
    protected void onCreate() {
        createdAt = LocalDateTime.now();
        updatedAt = LocalDateTime.now();
    }
     @PreUpdate
    protected void onUpdate() {
        updatedAt = LocalDateTime.now();
    }
}
```

Understand each field

id

private Long id;

Unique identification for every user.

Example:

1

2

3

The database automatically generates it because of:

@Id

@GeneratedValue(strategy = GenerationType.IDENTITY)

name

private String name;

Stores the user's name.

Example:

"Ravi Kumar"

email

private String email;

Stores the user's email.

@Column(nullable = false, unique = true)

means:

- nullable = false → email is required
- unique = true → duplicate email is not allowed

Example:

ravi@gmail.com

phone

private String phone;

Stores the user's phone number.

I recommend String instead of Long, because phone numbers can contain things like +91 and leading zeros.

Example:

9876543210

password

private String password;

Stores the password.

Important: In a real application, don't store the user's actual password as plain text. Later in the project, we should add **BCrypt password encryption**.

status

private String status;

Stores whether the user is active or inactive.

For example:

ACTIVE

INACTIVE

A new user could have:

ACTIVE

createdAt

private LocalDateTime createdAt;

Stores when the user was created.

Example:

2026-09-03T09:30:15

This is automatically populated by:

```
@PrePersist
protected void onCreate() {
   createdAt = LocalDateTime.now();
   updatedAt = LocalDateTime.now();
}
```

updatedAt

private LocalDateTime updatedAt;

Stores the last time the user's information was updated.

It automatically changes when the entity is updated:

@PreUpdate

protected void onUpdate() {

updatedAt = LocalDateTime.now();

}

Step 6 — Create User Repository

Now create:

repository/UserRepository.java

Code:

package com.company.userservice.repository;

import com.company.userservice.entity.User;

import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface UserRepository extends JpaRepository<User, Long> {

Optional<User> findByEmail(String email);

boolean existsByEmail(String email);

}

Why do we need Repository?

Repository communicates with the database.

Because we extend:

JpaRepository<User, Long>

Spring automatically gives us methods like:

save()

findById()

findAll()

deleteById()

For example:

```
userRepository.save(user);
```

stores a user in the database.

Step 7 — Create Service Layer

Create this file:

src/main/java/com/company/userservice/service/UserService.java

The Service Layer contains the **business logic**.

It will perform:

Create user

Get one user

Get all users

Update user

Delete user

UserService.java

```
package com.company.userservice.service;

import com.company.userservice.dto.UserRequest;

import com.company.userservice.entity.User;

import com.company.userservice.repository.UserRepository;

import org.springframework.stereotype.Service;

import java.util.List;

@Service
```

```
public class UserService {
    private final UserRepository userRepository;
   // Constructor injection
   public UserService(UserRepository userRepository) {
      this.userRepository = userRepository;
   }
   // 1. Create User
   public User createUser(UserRequest request) {
       // Check whether email already exists
      if (userRepository.existsByEmail(request.getEmail())) {
         throw new RuntimeException("Email already exists");
      }
      User user = new User();
      user.setName(request.getName());
      user.setEmail(request.getEmail());
      user.setPhone(request.getPhone());
      user.setPassword(request.getPassword());
      return userRepository.save(user);
   }
   // 2. Get User by ID
   public User getUser(Long id) {
       return userRepository.findById(id)
            .orElseThrow(() ->
                  new RuntimeException("User not found with id: " + id));
   }
```

```
    // 3. Get All Users
    public List<User> getAllUsers() {
        return userRepository.findAll();
    }
    // 4. Update User
    public User updateUser(Long id, UserRequest request) {
        User user = userRepository.findById(id)
            .orElseThrow(() ->
                new RuntimeException("User not found with id: " + id));
        user.setName(request.getName());
        user.setEmail(request.getEmail());
        user.setPhone(request.getPhone());
        user.setPassword(request.getPassword());
        return userRepository.save(user);
    }
   // 5. Delete User
    public void deleteUser(Long id) {
        User user = userRepository.findById(id)
            .orElseThrow(() ->
                new RuntimeException("User not found with id: " + id));
        userRepository.delete(user);
    }
}
```

Understand the Service

1. Create User

public User createUser(UserRequest request)

This receives user information from the Controller.

For example:

```
{
   "name": "Ravi",
   "email": "ravi@gmail.com",
   "phone": "9876543210",
   "password": "password123"
}
```

Then: User user = new User();

creates a User object.

Then we copy the values:

user.setName(request.getName());

user.setEmail(request.getEmail());

user.setPhone(request.getPhone());

user.setPassword(request.getPassword());

Finally: userRepository.save(user);

stores the user in the database.

2. Get User

public User getUser(Long id)

For example: GET /api/v1/users/1

The ID is 1.

code: userRepository.findById(id)

searches the database.

If the user exists, it returns the user.

If not: .orElseThrow(...)

throws an error.

3. Get All Users

```
public List<User> getAllUsers() {
   return userRepository.findAll();
}
```

This retrieves all records from the users table.

For example:

User 1

User 2

User 3

User 4

---

4. Update User

```
public User updateUser(Long id, UserRequest request)
```

First we find the existing user:

```
User user = userRepository.findById(id)
```

Then change the values:

```
user.setName(request.getName());
user.setEmail(request.getEmail());
user.setPhone(request.getPhone());
user.setPassword(request.getPassword());
```

Then save: userRepository.save(user);

So:

Existing User

↓

Find by ID

↓

Change information

↓

Save

↓

Database updated

5. Delete User

public void deleteUser(Long id)

First check that the user exists: User user = userRepository.findById(id)

Then: userRepository.delete(user);

removes the user from the database.

Step 8 — Create Controller

Now create: src/main/java/com/company/userservice/controller/UserController.java

The Controller is responsible for receiving HTTP requests.

UserController.java

```
package com.company.userservice.controller;

import com.company.userservice.dto.UserRequest;

import com.company.userservice.entity.User;

import com.company.userservice.service.UserService;

import jakarta.validation.Valid;

import org.springframework.http.HttpStatus;

import org.springframework.http.ResponseEntity;

import org.springframework.web.bind.annotation.*;
```

```
import java.util.List;
@RestController
@RequestMapping("/api/v1/users")
public class UserController {
        private final UserService userService;
    // Constructor injection
   public UserController(UserService userService) {
      this.userService = userService;
   }
   // CREATE USER
   @PostMapping
   public ResponseEntity<User> createUser(
         @Valid @RequestBody UserRequest request) {
       User user = userService.createUser(request);
       return new ResponseEntity<>(
            user,
            HttpStatus.CREATED
       );
   }
   // GET USER BY ID
   @GetMapping("/{id}")
   public ResponseEntity<User> getUser(
         @PathVariable Long id) {
          User user = userService.getUser(id);
       return ResponseEntity.ok(user);
   }
```

```
 // GET ALL USERS
    @GetMapping
    public ResponseEntity<List<User>> getAllUsers() {
            List<User> users = userService.getAllUsers();
            return ResponseEntity.ok(users);
    }
    // UPDATE USER
    @PutMapping("/{id}")
    public ResponseEntity<User> updateUser(
            @PathVariable Long id,
            @Valid @RequestBody UserRequest request) {
           User user = userService.updateUser(id, request);
          return ResponseEntity.ok(user);
    }
    // DELETE USER
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteUser(
            @PathVariable Long id) {
           userService.deleteUser(id);
          return ResponseEntity.noContent().build();
    }
}
```

Understand Controller

PostMapping

```
        @PostMapping
```

Used to create a new user.

Complete API:

POST http://localhost:8081/api/v1/users

GetMapping("/{id}")

@GetMapping("/{id}")

Used to get one user.

Example: GET http://localhost:8081/api/v1/users/1

Here:

1

is the user ID.

This: @PathVariable Long id

takes 1 from the URL.

GetMapping

@GetMapping

Used to get all users.

GET http://localhost:8081/api/v1/users

PutMapping("/{id}")

@PutMapping("/{id}")

Used to update an existing user.

Example: PUT http://localhost:8081/api/v1/users/1

DeleteMapping("/{id}")

@DeleteMapping("/{id}")

Used to delete a user.

Example: DELETE http://localhost:8081/api/v1/users/1

Step 9 — Test APIs

Successful Request

First create a valid user.

Postman

Method: POST

URL: http://localhost:8081/api/v1/users

Go to:

Body → raw → JSON

Send:

```
{
   "name": "Ravi Kumar",
   "email": "ravi@gmail.com",
   "phone": "9876543210",
   "password": "password123"
}
```

Expected result : HTTP 201 Created

Example response:

```
{
   "id": 1,
   "name": "Ravi Kumar",
   "email": "ravi@gmail.com",
   "phone": "9876543210"
}
```

Invalid Request

Now test validation.

Send:

```
{
   "name": "",
   "email": "wrong-email",
   "phone": "",
   "password": ""
}
```

Because your DTO contains: @NotBlank

and: @Email

Spring should reject the request.

The expected status is:400 Bad Request

However, to get a clean error response, let's create an exception handler.

Missing User

The task says: Missing user.

This means trying to retrieve a user that doesn't exist.

For example, suppose users are:

ID Name

1 Ravi

2 Suresh

Now request: GET http://localhost:8081/api/v1/users/100

There is no user with ID 100.

Your Service already contains:

```
public User getUser(Long id) {
        return userRepository.findById(id)
          .orElseThrow(() ->
                new RuntimeException(
                      "User not found with id: " + id
                ));
}
```

The response will be:

```
{
   "message": "User not found with id: 100"
}
```

Duplicate Email

The screenshot specifically asks you to test:

Duplicate email.

Suppose you already created:

```
{
   "name": "Ravi Kumar",
   "email": "ravi@gmail.com",
   "phone": "9876543210",
   "password": "password123"
}
```

Now try creating another user with the same email:

```
{
   "name": "Another User",
   "email": "ravi@gmail.com",
   "phone": "9999999999",
```

```
  "password": "abc123"
}
```

Your Service has:

```
if (userRepository.existsByEmail(request.getEmail())) {
  throw new RuntimeException("Email already exists");
}
```

So the API rejects it.

Create DuplicateEmailException

Create: exception/DuplicateEmailException.java

```
package com.company.userservice.exception;
public class DuplicateEmailException extends RuntimeException {
     public DuplicateEmailException(String message) {
      super(message);
  }
}
```

Change Service:

```
if (userRepository.existsByEmail(request.getEmail())) {
  throw new DuplicateEmailException(
        "Email already exists: " + request.getEmail()
  );
}
```

Add to GlobalExceptionHandler:

```
@ExceptionHandler(DuplicateEmailException.class)
public ResponseEntity<Map<String, String>> handleDuplicateEmail(
     DuplicateEmailException exception) {
```

```
    Map<String, String> error = new HashMap<>();
    error.put("message", exception.getMessage());
    return ResponseEntity
            .status(HttpStatus.CONFLICT)
            .body(error);
}
```

Now duplicate email gives: 409 Conflict

Example:

```
{
    "message": "Email already exists: ravi@gmail.com"
}
```

Invalid ID

This is slightly different from a missing user.

The task may expect you to test an invalid ID such as: GET /api/v1/users/abc

But your controller currently has: @PathVariable Long id

abc cannot be converted to Long.

Spring will automatically return: 400 Bad Request

For example: GET http://localhost:8081/api/v1/users/abc

You can improve the error message.

Add this to GlobalExceptionHandler:

Import
org.springframework.web.method.annotation.MethodArgumentTypeMismatchException;

Then:

```
@ExceptionHandler(MethodArgumentTypeMismatchException.class)
public ResponseEntity<Map<String, String>> handleInvalidId(
        MethodArgumentTypeMismatchException exception) {
```

```
    Map<String, String> error = new HashMap<>();
    error.put("message", "Invalid user ID. ID must be a number.");
    return ResponseEntity
            .status(HttpStatus.BAD_REQUEST)
            .body(error);
}
```

Now: GET /api/v1/users/abc

returns:

```
{
    "message": "Invalid user ID. ID must be a number."
}
```

Complete Testing Table

Test	Request	Expected
Successful request	POST /api/v1/users with valid data	201 Created
Invalid request	POST /api/v1/users with empty/invalid fields	400 Bad Request
Missing user	GET /api/v1/users/100	404 Not Found
Duplicate email	POST /api/v1/users with existing email	409 Conflict
Invalid ID	GET /api/v1/users/abc	400 Bad Request

Step 10 — Documentation

README.md

User Service - Spring Boot Microservice

1. Project Purpose

User Service is a Spring Boot based microservice that provides REST APIs

for managing users.

The service supports basic CRUD operations:

- Create User
- Get User
- Get All Users
- Update User
- Delete User

The application uses Spring Data JPA for database communication and PostgreSQL as the database.

2. Technologies

- Java 17 / Java 21
- Spring Boot
- Spring Web
- Maven
- Validation
- Spring Boot Actuator
- Postman

3. Project Structure

```
src/main/java/com/company/userservice
 ├── controller
 │    └── UserController.java
 ├── service
 │    └── UserService.java
```

```
├── repository
│       └── UserRepository.java
├── entity
│       └── User.java
├── dto
│       ├── UserRequest.java
|   └── UserResponse.java
├── exception
│       ├── UserNotFoundException.java
│       ├── DuplicateEmailException.java
│       └── GlobalExceptionHandler.java
├── config
└── UserServiceApplication.java
```

API List

Create User

POST

/api/v1/users

Example request:

```
{
   "name": "Ravi Kumar",
   "email": "ravi@gmail.com",
   "phone": "9876543210",
   "password": "password123"
}
```

Get User

GET

/api/v1/users/{id}

Example:

/api/v1/users/1

Get All Users

GET

/api/v1/users

Update User

PUT

/api/v1/users/{id}

Example request:

```
{
   "name": "Ravi Kumar Updated",
   "email": "raviupdated@gmail.com",
   "phone": "9999999999",
   "password": "newpassword"
}
```

Delete User

DELETE

/api/v1/users/{id}

Example:

/api/v1/users/1

How to Run

Step 1

Clone or download the project.

Step 2

Open the project in IntelliJ IDEA, Eclipse or VS Code.

Step 3

Configure PostgreSQL.

Step 4

Create the database:

CREATE DATABASE usersdb;

Step 5

Update the PostgreSQL username and password in:

application.properties

Step 6

Run:

UserServiceApplication.java

Step 7

The application starts on:

http://localhost:8081